

Causal Data Integration

Brit Youngmann
CSAIL MIT
brity@mit.edu

Michael Cafarella
CSAIL, MIT
michjc@csail.mit.edu

Babak Salimi
University of California, San Diego
bsalimi@ucsd.edu

Anna Zeng
CSAIL, MIT
annazeng@mit.edu

ABSTRACT

Causal inference is fundamental to empirical scientific discoveries in natural and social sciences; however, in the process of conducting causal inference, data management problems can lead to false discoveries. Two such problems are (i) not having all attributes required for analysis, and (ii) misidentifying which attributes are to be included in the analysis. Analysts often only have access to partial data, and they critically rely on (often unavailable or incomplete) domain knowledge to identify attributes to include for analysis, which is often given in the form of a causal DAG. We argue that data management techniques can surmount both of these challenges. In this work, we introduce the Causal Data Integration (CDI) problem, in which unobserved attributes are mined from external sources and a corresponding causal DAG is automatically built. We identify key challenges and research opportunities in designing a CDI system, and present a system architecture for solving the CDI problem. Our preliminary experimental results demonstrate that solving CDI is achievable and pave the way for future research.

PVLDB Reference Format:
Brit Youngmann, Michael Cafarella, Babak Salimi, and Anna Zeng. Causal Data Integration. PVLDB, 16(10): 2659 - 2665, 2023.
doi:10.14778/3603581.3603602

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/brityoungmann/CDI/blob/main/README.md.

1 INTRODUCTION

Causal inference lies in the heart of empirical research in natural and social sciences and is commonly used in multiple disciplines, including sociology, medicine, and economics [26, 33]. It aims to answer causal queries, such as, "Does being overweight cause coronary heart disease – independent of cholesterol, and diabetes?" or "Do tobacco advertisements entice adolescents to buy more cigarettes regardless of whether their parents smoke?" Causal inference enables analysts to answer causal questions about attributes from a dataset, ultimately enabling them to make real-world discoveries. Pearl's framework [40], which we adapt in this work, provides a principled way to causal inference using structural causal models.

Cause-effect questions are designed to determine whether an *exposure* variable (a pain reliever) causes or affects an *outcome* variable (pain). To correctly estimate causal effects, one must consider *confounding variables*—variables that influence both the exposure and outcome and thus might distort the association between them. Otherwise, one might draw perplexing conclusions due to *confounding bias* [41] that can lead to overestimation or underestimation of the association between exposure and outcome. The internal validity of a study heavily relies on how well confounding bias has been addressed. However, to determine a sufficient set of confounding variables to account for confounding bias, *background knowledge* about the data generative process is required. This knowledge is often given in the form of a *causal DAG* depicting causal relationships between the attributes [40] (and is wildly used in econometric and social sciences [26, 33]). Pearl [40] presented sufficient and necessary conditions for identifying[1] the *adjustment set* of variables to include in the analysis, which can be checked against a causal DAG. However, analysts often lack such a DAG [22]. While associational assumptions are testable from data, causal relations cannot, in general, be fully recovered from data [41].

Table 1: Example Input Dataset.

State	Mask Policy	Confirmed Cases	New Cases	Recovered	Death cases
MA	yes	121046	2740	4980	109
FL	yes	640978	24349	25140	55
CA	no	735235	31150	42170	34
SD	no	15300	1791	2083	49

We identify two fundamental challenges for conducting valid causal inference: First, critical confounding variables may not be included in the data. Second, even if they are included, the background knowledge required to identify the correct adjustment set of variables might be missing. We argue that data management techniques and ideas can surmount both of these challenges. We illustrate that via the following example:

Mary, a data analyst in the WHO organization, aims to estimate the (treatment) effect of a mask policy on the coronavirus mortality rate. To do this, she examines a dataset containing Covid-19-related facts in multiple states in the US (an illustration of this dataset is given in Table 1), and uses a Causal analysis tool (e.g., [49, 53]). To correctly estimate the effectiveness of face masks, Mary must consider confounding variables. However, there are confounding variables that are not included in her data. For example, the weather affects people's willingness to wear masks and Covid-19 death rate [47]. Moreover, to determine a sufficient set of confounding variables, Mary must use background knowledge of causal relationships she may lack.

In this example, variables missing from the dataset are critical for the analysis to avoid confounding bias. Thus, Mary must *integrate her data* with external data sources. This is a laborious task, typically done using data discovery and integration tools (e.g., [36, 59]) by skilled programmers—which is often not the case for social or

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 10 ISSN 2150-8097.
doi:10.14778/3603581.3603602

[1] In causal inference, identifiability typically refers to the conditions that permit measuring causal effect from observed data. Here, we discuss the ability to identify a sufficient adjustment set of variables for accurate causal inference.

natural scientists. We argue this is the case for many real-life scenarios. After augmenting the data to include unobserved variables, the next step is determining a sufficient adjustment set of variables to account for confounding bias. However, Mary does not have the required background knowledge to do that.

Our Vision: Our goal is to mine unobserved confounding variables and missing background knowledge needed to answer causal questions. To this end, we introduce the Causal Data Integration (CDI) problem. CDI is an effort to augment an input dataset with unobserved variables and background knowledge required for causal analysis. To address the challenge of finding unobserved variables, the system mines variables from external sources. To overcome the challenge of identifying the right adjustment set of confounders, the system then builds a corresponding causal DAG, ensuring the adjustment set is sufficient for controlling the confounding bias. A CDI system may greatly reduce the manual effort devoted to performing causal analysis, allowing analysts to get all required data and background knowledge quickly and effectively.

CDI may also be beneficial for the data management community. Causal inference has been shown to be useful for various data management tasks, including query result and classifier explanation [8, 42, 56], and hypothetical reasoning [21]. However, some of these studies make strong assumptions about the underlying causal DAG that may not hold in practice [26, 33]. For example, [21] assumes that the DAG is known, and [42] assumes that all confounding variables are present in the input dataset. To address these issues, a CDI system can be utilized to construct a causal DAG for a given dataset while integrating relevant missing attributes. This not only addresses the limitations of existing studies but also enables the use of causal inference in more complex and realistic scenarios.

In this work, we show how a CDI system can be accomplished.

Measure of Success: Distinct from the classical data integration task, where success is defined by its ability to combine data from different sources and provide users with a unified materialized view of them, the success of a CDI system is defined by the ability to discover all relevant unobserved variables and recover the correct set of confounders for a causal question. The CDI task is potentially easier than data integration. However, its impact is greater, as it enables analysts to conduct causal analysis more easily. A CDI system enables analysts to include all confounding variables in the analysis; failing to do so may lead to false discoveries, incorrect medical diagnoses, and erroneous conclusions [26, 33]. In our example, a CDI system mines, among others, attributes describing the weather and population density per state, and infers causal relations among the attributes in the augmented dataset. A successful CDI system would assist Mary in identifying the right adjustment set of attributes to account for confounding bias, allowing her to accurately estimate the effect of an enforced mask policy on the mortality rate.

Challenges & Opportunities: The first challenge is to augment an input dataset with unobserved variables, ensuring the *output causal DAG is complete* (i.e., there are no unobserved confounding variables). Data discovery methods have been extensively researched to identify relevant data sources that can be integrated with an input dataset for attribute extraction [12, 14]. However, these methods are not specifically designed to discover unobserved attributes that are relevant for cause-effect estimation. This presents a unique opportunity to develop dedicated data discovery tools tailored for identifying unobserved confounding variables.

Extracted attributes may contain data quality issues, such as outliers and missing values. These issues may lead to erroneous conclusions, compromising the reliability of causal inference. In particular, missing values pose a significant challenge in causal inference as they may result in *selection bias* [10], where certain samples are preferentially excluded from the data. Other data quality issues, such as inconsistent data formats and data duplication, can also impact the accuracy of causal inference analysis. Therefore, in addition to handling missing values, it is crucial to ensure that the system is robust to various data quality issues to ensure the validity of causal inference results.

The challenge of identifying the correct adjustment set and constructing a causal DAG is particularly difficult when dealing with hundreds of extracted attributes. A human-driven causal DAG construction (from domain expertise) is infeasible in this scenario, and automatic methods for building a full causal DAG may not be feasible either [22]. To this end, we propose the construction of a cluster causal DAG (C-DAG) [9], which groups related attributes and only specifies causal relationships between clusters of attributes. By doing so, the correct adjustment set of attributes can still be identifiable from this C-DAG. This approach reduces cognitive overhead for analysts and makes causal analysis possible for high-dimensional data, which is currently not feasible for existing tools (e.g., [49, 53]). However, a main challenge is how to maintain the necessary level of granularity to ensure accurate causal inference. The limitations of existing solutions and challenges in constructing the desired C-DAG are discussed, and a limited prototype implementation for solving the CDI problem is presented with preliminary experimental results demonstrating that solving CDI is achievable.

2 THE CDI PROBLEM

2.1 Data Model and Background

The input dataset $\mathcal{D}$ contains the exposure (T) and outcome (O) attributes. A *causal DAG* for $\mathcal{D}$ is a DAG whose nodes are the attributes and whose edges capture all causal relationships between the attributes [40]. Generally, because causes must precede effects, the literature considers causal DAGs rather than graphs [40]. Causal DAGs provide a simple way of graphically representing causal relationships and are particularly helpful in understanding potential sources of bias in causal estimations. It is well-known that background knowledge is required to determine a causal DAG for a given dataset [41]. A causal DAG can only be as good as the background information used to create it [38]; a DAG is complete and therefore has a causal interpretation only if it contains all common causes of any two variables. In Section 3.3, we review existing approaches for building a causal DAG for a given dataset and discuss their limitations. We assume the analyst is interested in investigating a causal relationship between T and O. She may wish to estimate, e.g., *the direct, indirect, or total effect* [40]. The total effect of T on O is the total extent to which O is changed by the T. It is equal to the sum of the direct and indirect effects. Causal questions about total and (in)direct effect could be answered by identifying the right set of *confounding* (that influence both T and O) and *mediating* (that

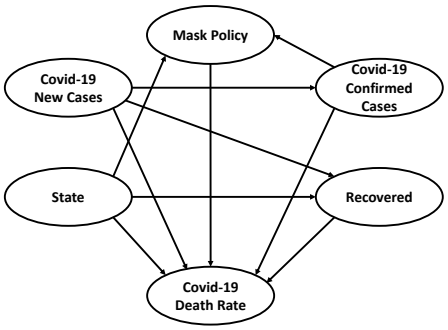


Figure 1: Example causal DAG for Table 1

mediate the relationship between T and O) variables. Such variables could be identified using graphical criteria (e.g., the backdoor criterion [40]) that can be checked against a causal DAG.

Example 2.1. In our example, T is `mask policy`, and O is `death cases`. A corresponding causal DAG is shown in Figure 1. To obtain a reliable estimate of the total effect of T on O, it is crucial to adjust for all relevant confounding variables. However, mistakenly assuming that the DAG contains all relevant information, the adjustment set only includes the attributes `state` and `confirmed cases`. As is common in causal inference, missing confounders can lead to biased estimates. Thus, it is necessary to consider external data to incorporate missing variables.

2.2 Problem Formulation & Challenges

Given an input dataset, the system mines unobserved confounding attributes relevant for a given causal estimation, yielding an augmented dataset $\widehat{\mathcal{D}}$. It then maps the attributes in $\widehat{\mathcal{D}}$ into a causal DAG, to enable analysts to determine the adjustment set. We next outline key challenges in designing such a system:

(1) Completeness: Unobserved variables can dramatically affect the quality of causal analysis [32, 41]. Thus, the first challenge is to *ensure the generated DAG is complete*, i.e., there are no unobserved confounding variables w.r.t. the provided data sources. Our goal is to ensure that all relevant information can be automatically extracted from the provided sources. However, completeness, in general, cannot be guaranteed since relevant variables may exist outside of the provided data sources.

(2) Robustness: The robustness of a CDI system is determined by its ability to handle various data quality issues that may arise during the integration process. As in general data integration, there are multiple data quality issues that a CDI system has to face, including missing values and outliers. The impact of such issues on causal inference is particularly significant. Unlike in machine learning, where data quality issues may only result in a slight drop in accuracy, these issues can lead to false discoveries in causal inference if not handled properly. This is because if these issues are not randomly distributed across the exposure groups, they can significantly impact the validity of the resulting conclusions. One common issue that can significantly impact causal inference is missing data. Extracted attributes may contain missing values, which can lead to selection bias and erroneous conclusions. Thus, a key challenge is to ensure the system is robust to various data quality issues.

Table 2: Extracted attributes from different sources.

Extracted Attributes from US Open Data

State	Population size	Population density
MA	6,981,974	901
FL	22,244,823	402
CA	39,029,342	254
SD	909,824	12

Extracted Attributes from DBPedia

State	Governor	snow inch	Avg temp.	Min temp.
MA	Maura Healey	51.05	48.14	23
FL	Ron DeSantis	-	71.8	70
CA	Gavin Newsom	-	61.17	54
SD	Kristi Noem	37.43	45.54	34

(3) Conciseness: We can potentially extract hundreds of attributes from given data sources, which can lead to a complex, high-dimensional causal DAG. To overcome this, the CDI system should aim to reduce users' cognitive overhead in interpreting the DAG. It is also crucial to ensure the scalability of causal analysis, as existing tools are not feasible for high-dimensional data [49, 53]. One approach is to ensure that related attributes are grouped into clusters while still ensuring that the correct set of confounders is identifiable. Thus, a third challenge is to ensure the output DAG is concise. The CDI system should be able to construct a causal DAG that effectively captures the causal relationships between attributes while maintaining the necessary level of granularity to ensure accurate causal inference. This will enable analysts to efficiently handle high-dimensional data while facilitating information elicitation about the validity of the assumptions used for drawing causal inferences.

3 THE CDI SYSTEM

Our system operates as a 3-steps pipeline. First, the *Knowledge Extractor*, extracts candidate attributes to handle the *completeness* challenge. The *Data Organizer* then handles data quality issues to account for the *robustness* challenge. Last, the *Causal DAG Builder* builds a clustered casual DAG to handle the *conciseness* challenge. We next present the key challenges and research opportunities for each system component.

3.1 The Knowledge Extractor

This module receives a table $\mathcal{D}$ (containing T and O) and sources of knowledge. From each source, it extracts attributes representing additional properties of entities from $\mathcal{D}$. A successful Knowledge Extractor mines all unobserved variables that should be included in the analysis. The primary goal of the *Knowledge Extractor* is to extract all relevant variables from the provided data sources to enable reliable causal inference. In causal inference, the *unconfoundedness assumption* [39] (the assumption that all variables affecting both the exposure T and the outcome Y are observed and can be controlled for) is a key consideration. This assumption is similar to the closed-world assumption in database in that it assumes that the database is complete. However, since complete knowledge of all variables affecting both T and O is often unattainable, the focus should be on extracting all relevant variables from the provided data sources. The module is designed to ensure that as much relevant information as possible can be automatically extracted from the provided sources, and its capabilities can be expanded over time to incorporate new sources or refine the extraction and integration process. We next provide an example of how it works with specific data sources.

Data Lakes: There has been unprecedented growth in the volume of publicly available data (e.g., the US Federal Open Data [6] and the



Discovering Similarity Inclusion Dependencies

YOURI KAMINSKY*, Hasso Plattner Institute, University of Potsdam, Germany
EDUARDO H. M. PENA, Federal University of Technology – Paraná, Brazil
FELIX NAUMANN, Hasso Plattner Institute, University of Potsdam, Germany

Inclusion dependencies (INDs) are a well-known type of data dependency, specifying that the values of one column are contained in those of another column. INDs can be used for various purposes, such as foreign-key candidate selection or join partner discovery. The traditional notion of INDs is based on clean data, where the dependencies hold without exceptions. Unfortunately, data often contain errors, preventing otherwise valid INDs from being discovered. A typical response to this problem is to relax the dependency definition using a similarity measure to account for minor data errors, such as typos or different formatting. While this relaxation is known for functional dependencies, for inclusion dependencies no such relaxation has been defined.

We formally introduce *similarity inclusion dependencies*, which relax the inclusion by demanding the existence only of sufficiently similar values. Similarity inclusion dependencies can fulfill traditional IND use cases, such as foreign-key candidate discovery, even in the presence of dirty data. We present Sawfish, the first algorithm to discover all similarity inclusion dependencies in a given dataset efficiently. Our algorithm combines approaches for the discovery of traditional INDs and string similarity joins with a novel sliding-window approach and lazy candidate validation. Our experimental evaluation shows that Sawfish can outperform a baseline by a factor of up to 6.5.

CCS Concepts: • **Information systems** → **Information integration**; *Data mining*.

Additional Key Words and Phrases: data profiling, data lake, joinability

ACM Reference Format:
Youri Kaminsky, Eduardo H. M. Pena, and Felix Naumann. 2023. Discovering Similarity Inclusion Dependencies. *Proc. ACM Manag. Data* 1, 1, Article 75 (May 2023), 24 pages. https://doi.org/10.1145/3588929

1 INCLUSION DEPENDENCY

Data profiling is the process of extracting metadata from datasets. Data dependencies are an important type of metadata and, thus, have a crucial role in data profiling. There are different forms of data dependencies, e.g., functional dependencies (FDs) and inclusion dependencies (INDs). In particular, INDs express that the tuples of one column-combination are contained in the tuples of another column-combination. We call an IND unary, if it holds between two individual columns. INDs help data practitioners to understand and structure unknown data, in particular, in discovering foreign key candidates and joinable partners [20]. Moreover, INDs have assisted schema design in [15] and can improve query execution [12].

*Corresponding author (youri.kaminsky@hpi.de)

Authors' addresses: Youri Kaminsky, Hasso Plattner Institute, University of Potsdam, Germany, youri.kaminsky@hpi.de; Eduardo H. M. Pena, Federal University of Technology – Paraná, Campo Mourão, Paraná, Brazil, eduardopena@utfpr.edu.br; Felix Naumann, Hasso Plattner Institute, University of Potsdam, Germany, felix.naumann@hpi.de.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART75 $15.00
https://doi.org/10.1145/3588929

Traditional IND discovery assumes clean data: *all* tuples of the dependent column-combination must be exactly *equal* in the referenced column-combination. However, the ever-increasing volume of data also leads to more "dirty" data [18]. Thus, relaxed dependencies have been introduced to deal with erroneous data [4].

One example of relaxed dependencies is matching dependencies (MDs), which generalize the concept of functional dependencies (FDs) [23]. For a traditional FD, the tuples on the dependent side must be equal for all equal tuples on the determinant side. In contrast, MDs use similarity measures instead of the strict equality constraint. In other words, an MD holds if, for all *similar* tuples on the determinant side, all tuples on the dependent side are also *similar*. MDs allow data practitioners to address typical use cases of FDs, such as schema normalization, even in the presence of dirty data. Moreover, MDs can also be used for duplicate detection [23]. A feature of MDs is that they support arbitrary similarity measures and a configurable similarity threshold to balance the error tolerance and over generalization.

There is no corresponding notion yet for INDs to allow for similar values. Thus, we introduce similarity inclusion dependencies (sINDs). In contrast to traditional INDs, sINDs use a similarity measure to define inclusion. An sIND holds if, for all dependent values, there exists a referenced value that is at least *similar*. Like MDs, sINDs support arbitrary similarity measures and configurable similarity thresholds. For this work, we consider representatives of both edit-based and token-based similarity measures: the edit distance and the Jaccard similarity.

sINDs are a natural way to handle dirty data. Besides traditional IND use cases, sINDs can also identify sources of possibly erroneous data in data lakes. If a candidate IND does not hold, but its respective sIND holds, the data can be analyzed and used to identify erroneous relations. Either only one of the relations contains errors, so we can use the other relation to fix these, or both relations contain errors, and the data needs to be cleaned more thoroughly.

To illustrate the usefulness of sINDs, we present an example in Table 1. The tables are for a fictitious soccer tournament. While one table shows the results after all teams played against each other, the other table presents an aggregation of the participation forms of all goalkeepers in the tournament. We would assume that the values of the *club* column of Table 1b are contained in the values of the *name* column of Table 1a. However, multiple goalkeepers made minor spelling mistakes in their club name. Despite these mistakes, we want to discover the dependency. On the one hand, we are interested in joining these columns to know which goalkeeper played for the most successful team. On the other hand, we could perform a data cleaning task, because all mistakes are only in the *club* column of Table 1b. Thus, there is a matching counterpart in the other column to automatically correct the errors.

Table 1. Example relations of a soccer tournament

(a) Final Results

results	
name	points
SpVgg Beelitz	4
Potsdamer SC	9
SpVgg Bernau	4
VfL Potsdam	0

(b) Participating Goalkeepers

goalie	
p_id	club
202	SpVgg Beelitz
216	SpVgg Beelittz
469	Potsdamer SC
617	SpVgg Be_nau
692	ViL Potsdam
853	Potsdamer SC

There are other known extensions to INDs that can deal with dirty data. One example is partial INDs [2], which allow a certain portion of the dependent tuples to not be present in any form in the referenced tuples. Therefore, partial INDs are especially useful when dealing with incomplete columns. However, partial INDs provide no insight into what kind of matching failures occur. If a significant portion of tuples have a similar, but not equal counterpart, no partial INDs would be found. Since half of the goalkeepers in our example have misspelled their club names, we would need to set the error threshold above 50% to find a partial IND. In a typical database, this threshold would lead to many spurious dependencies. A special form of partial dependencies are conditional INDs (cINDs), which hold only for tuples that fulfil a certain condition [3, 1].

Another example are approximate INDs [13]. They are discovered on a sample of the complete data and show that an IND holds only with a certain probability. This relaxation is used to increase the discovery speed, but not the generality of the found IND. Moreover, approximate INDs also provide no insight into data errors. In our example, typical sampling strategies would lead to different value sets for the columns. Therefore, we would not find a dependency in our samples.

Apart from introducing the concept of similarity inclusion dependencies, we present **Sawfish**, the first approach to efficiently discover sINDs in a given dataset. In particular, we make the following contributions:

(1) We introduce the formal concept of similarity inclusion dependencies.
(2) We propose Sawfish, an efficient approach to discover all unary sINDs in a given dataset using the edit-distance and the Jaccard similarity measure.
(3) We offer an in-depth evaluation of Sawfish, comparing our approach to the state-of-the-art IND discovery algorithm Binder [22] and a naive baseline. We show how Sawfish scales with the size of the input data and different algorithm configurations.
(4) We present a case study that explores the usefulness of the discovered sINDs. We have manually built a ground-truth with over 1000 sINDs and publicly provide the annotations.
(5) We integrate Sawfish on the Metanome data profiling platform [21], so it can be easily used with a variety of datasets and compared to other data profiling algorithms. All code is made publicly available.

The remainder of this work is structured as follows. In Section 2 we present related research on inclusion dependencies, other relaxed dependencies and string similarity joins. We formally define sINDs in Section 3. Section 4 shows how to efficiently discover sINDs and presents the general principle of Sawfish. We present an exhaustive evaluation of Sawfish in Section 5. Finally, Section 6 draws a conclusion and gives insights into the limitations of Sawfish and provides an outlook on future work.

2 RELATED WORK

Although there has been no research on similarity inclusion dependencies yet, three research areas overlap with this work: traditional inclusion dependency discovery, other relaxations of dependencies, and string similarity joins.

Inclusion dependencies (INDs) are a well-known type of data dependency and have several algorithms to discover them from data— see Dürsch et al. for an overview of approaches [9]. There are approaches for both unary and n-ary IND discovery. De Marchi et al. presented an early algorithm to discover unary INDs by intersecting the attribute sets for each value [6]. Many also discovers unary INDs, but focuses on the problem of finding INDs in millions of tables [24]. A prominent representative for n-ary, i.e., inclusion between tuples of attribute lists, IND discovery is Zigzag [7]. It combines both up- and downwards pruning to discover maximal n-ary INDs efficiently.



Effective Entity Augmentation By Querying External Data Sources

Christopher Buss
Oregon State University
bussch@oregonstate.edu

Jasmin Mousavi
Oregon State University
mousavij@oregonstate.edu

Mikhail Tokarev
Oregon State University
tokarevm@oregonstate.edu

Arash Termehchy
Oregon State University
termehca@oregonstate.edu

David Maier
Portland State University
maier@pdx.edu

Stefan Lee
Oregon State University
leestef@oregonstate.edu

ABSTRACT

Users often want to augment and enrich entities in their datasets with relevant information from external data sources. As many external sources are accessible only via keyword-search interfaces, a user usually has to manually formulate a keyword query that extract relevant information for each entity. This approach is challenging as many data sources contain numerous tuples, only a small fraction of which may contain entity-relevant information. Furthermore, different datasets may represent the same information in distinct forms and under different terms (e.g., different data source may use different names to refer to the same person). In such cases, it is difficult to formulate a query that precisely retrieves information relevant to an entity. Current methods for information enrichment mainly rely on lengthy and resource-intensive manual effort to formulate queries to discover relevant information. However, in increasingly many settings, it is important for users to get initial answers quickly and without substantial investment in resources (such as human attention). We propose a progressive approach to discovering entity-relevant information from external sources with minimal expert intervention. It leverages end users' feedback to progressively learn how to retrieve information relevant to each entity in a dataset from external data sources. Our empirical evaluation shows that our approach learns accurate strategies to deliver relevant information quickly.

PVLDB Reference Format:
Christopher Buss, Jasmin Mousavi, Mikhail Tokarev, Arash Termehchy, David Maier, and Stefan Lee. Effective Entity Augmentation By Querying External Data Sources. PVLDB, 16(11): 3404 - 3417, 2023.
doi:10.14778/3611479.3611535

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/bussch/QueryBasedEntityAugmentation.

1 INTRODUCTION

There is a recognized need to collect and connect information from a variety of data sources [14, 18, 23]. As an example, we have recently worked in a large-scale NIH-funded project to augment the information of biomedical entities by querying other biomedical data sources [49]. The main focus of this project is to *repurpose* current drugs to treat or mitigate the symptoms of diseases for which there is insufficient time or resources to develop effective treatments (e.g., new or rare diseases) [2]. Biomedical researchers often have some local dataset of available drugs (e.g., a dataset of *FDA-approved uses* of drugs). Given a drug in the local dataset, a researcher usually needs to query external data sources to find additional information about the drug (e.g., its *off-label uses*).

Due to a lack of access or resources, external information often must be retrieved through querying [14, 47]. Many data sources are only accessible via query interfaces or APIs. Even with access, it may require too much of a resource (e.g., storage space, time) to download and maintain an up-to-date copy of the external dataset. Thus, information relevant to some local entity must often be gathered on a as-needed basis by querying external data sources. For example, as many biomedical data sources are available only via query APIs, the users of the aforementioned drug repurposing data collection system must often query the information relevant to their current drug of interest through query APIs.

However, formulating a query that extracts specific information can be troublesome. Different data sources often represent the same concept in distinct forms [12, 15] such that one needs to tailor their query to specific external data sources. Figure 1 illustrates a case where users have a local dataset of FDA-approved uses of drugs, named *FDA-Approved Uses*, and would like to query an external data source that contains the off-label uses of those drugs, named *Off-Label Uses*. A drug that is identified by one of its brand names (e.g., *Zoloft*) in *FDA-Approved Uses* is referred to by its generic name (e.g., *Sertraline*) in *Off-Label Uses*. Because of heterogeneities, one may not know how to query for a specific external entity prior to investigating the content and structure of the data in the external source. Consider a biomedical researcher who seeks additional information about the drug *Zoloft* in their local dataset. Since they are only aware of the structure and content of their local dataset, they query the external data source for *Zoloft*, but that elicits no results. They try again using a much more general description of *Zoloft* (i.e., being a *serotonin reuptake inhibitor*). However, their under-specified query produces many results, most of which are irrelevant (i.e., contain information about drugs that are not *Zoloft*). After additional trial-and-error, they find a query that retrieves *Sertraline*. More work is required to then merge the local and external entities into one coherent representation.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 11 ISSN 2150-8097.
doi:10.14778/3611479.3611535

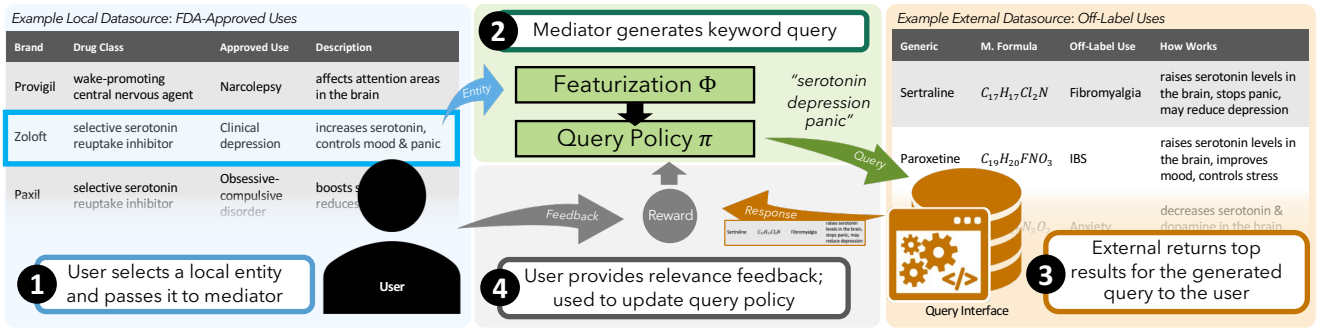


Figure 1: An example of our framework for a single user and single external data source. The user selects (by query, GUI, etc,.) the local entity *Zoloft*. The mediator uses its learned query policy to extract the relevant entity (*Sertraline*) from the external source. The user provides relevance feedback on the results, which is then used to further refine the mediator's querying policy.

Manually querying for specific external entities takes too much time and financial resources. Continuing our example, if the researcher needs additional information for another drug in their local dataset, they will need to repeat the entire process. Moreover, if they need information from additional external data sources, then the work required to query for each drug is greatly exacerbated. Furthermore, any other researcher with a similar information need must *repeat the same such work themselves.*

To alleviate the burden, one can use a **shared system that automates query formulation**. This **mediator system** acts as a go-between for users and external data sources: a user specifies a local entity (e.g., *Zoloft*) perhaps through a query or a graphical user interface, and the mediator maps the local entity to queries that retrieve the relevant external entities (e.g., *Sertraline*) from their respective external sources.

To the best of our knowledge, such mediators are currently created by *manually writing programs* that generate queries for specific external sources to retrieve relevant records to a given local entity. Each program implements a set of manually written rules specific to its external source. These rules *cannot* necessarily be reused across data sources. Thus, the mediator requires a significant amount of labor and expert attention to build and maintain. Instead of conducting their own research, biomedical researchers in our NIH-funded project spend most of their time writing these programs and investigating the content and structure of every external source to ensure that the programs formulate appropriate queries.

In this paper, we examine methods for *learning the mediator's query policy online* through user interaction. As illustrated in Figure 1, after the user specifies a local entity, the mediator formulates a query to retrieve records from an external source according to its *query policy* and shows the returned external records to the user. The user then provides feedback on the relevance of the returned records to the local entity. Our mediator learns to revise its query policy and improves its performance using the user's feedback.

An alternative to this online learning paradigm is to use offline training data to learn query formulation but collecting and labeling such data still requires considerable manual effort from domain experts [14]. Particularly, it is challenging to gather useful training data from external sources. The data collection/labeling might need to be repeated as the external datasets evolve. In many domains (e.g., drug repurposing for emerging viral diseases), users cannot wait long to prepare offline training data.

Of course, online learning of query policies comes with its own set of challenges. First, the mediator should learn to formulate reasonably accurate queries over external sources early on. We assume the mediator must be effective in the *short run* so users will continue to provide feedback. It is particularly difficult to meet this goal over large local or external datasets as the amount of required feedback for accurate learning generally grows with the number of entities. Second, the mediator should improve its querying policy and increase the effectiveness of its results in the *long run*. Online learning literature indicates that a policy that is effective in the *short run* (i.e., meets the first challenge) might not be accurate in the *long run* as it might become biased to early observations or decisions that do not deliver accurate results in the *long run* [42]. Third, due to lack of prior knowledge about the precise content and structure of the relevant external information, the number of candidate queries for a local entity might be enormous. This *large search space* makes finding effective queries difficult.

Due to the wide-spread use of keyword query interfaces over external sources, we develop online learning methods for formulating keyword queries. There are systems for automatic keyword query formulation, but they assume returned results are always relevant, which is not usually true and is the challenge that we address [47] (see Section 8). Our contributions are as follows:

- We present a framework for on-demand collection of relevant external entities only accessible via query interfaces (Section 2).
- We define the problem of online query-policy learning within the context of the aforementioned framework (Section 3).
- We present a method that learns a separate query policy for each individual local entity. We show that this approach does not scale to large local datasets as it might require a great deal of user feedback (Section 4).
- We propose an entity-conditional method that learns a query policy jointly over all local entities. This approach significantly reduces the amount of user feedback required to learn effective query policies. To overcome representational heterogeneity across the local and external sources, we propose techniques to use features and keywords from the external results in our model and queries, respectively (Section 5).
- If the local dataset contains many diverse entities, it might not be possible to learn effective queries for many entities using a shared model. Hence, we propose an approach that gradually

replaces a shared model with entity-specific ones based on the effectiveness of the shared model. The resulting models will retain the desirable properties of the shared model in the *short run* and learn more effective queries in the *long run* (Section 6).

- We explore whether the broad language understanding capabilities of state-of-the-art large language models can improve query generation. We train a neural network over features extracted by large-scale pretrained Longformer [5] and LLaMA [44] models to serve as our query policy (Section 6).
- We perform extensive empirical studies using six pairs of real-world datasets from different domains, including biology, products, and news. Our studies indicate that our proposed methods learn reasonably effective queries quickly and improve their accuracy in the long run over large datasets (Section 7).

2 GENERAL FRAMEWORK

The **mediator** wraps the local dataset and the query interface over the external data source. We assume the mediator has full access to the local dataset, but can only access external datasets through their query interfaces. Given a user-specified entity from the local dataset, the mediator must devise and submit a query to the interface to extract external entities relevant to the given local entity. This framework is not tied to a particular method by which a user specifies the local entity (e.g., through query or GUI).

Local Dataset. To simplify our exposition, we assume the local dataset is a single relational table where each tuple stores information about a distinct entity. One may extend our approach to multi-relational datasets by defining an entity as the join of its related tuples. We denote the set of local dataset entities as $\mathcal{E}$.

External Dataset. Like the local dataset, we model the external dataset as a set of entities (i.e., tuples). Given local entity e and external dataset D, $X(e) \in D$ represents the external entity that is relevant to the local one. The definition of "relevant entity" depends on the domain. For example, a clinical trial is relevant to the drug that it concerns. For notational convenience, we assume only one relevant external entity exists for each local entity, however, in the case of more than one, we can easily extend $X(e)$ to be the set of all relevant entities. If no relevant entities exist, then extracting $X(e)$ is impossible regardless of the method used. Thus, to accurately evaluate our methods, we assume that $X(e)$ always exists.

Example 1. Figure 1 shows excerpts of a local (left) and an external (right) dataset. $\mathcal{E}$ consists of all drugs in *FDA-Approved Uses*. If e is *Zoloft* then the relevant tuple $X(e)$ in *Off-Label Uses* is *Sertraline*. We show the content of $X(e)$ for explanation's sake. In a real setting, the content of $X(e)$ would not be known a priori.

Querying Policy. We call the queries submitted by the mediator to the external data source *mediator queries*. We denote the set of *all possible* mediator queries as $\mathcal{Q}$. $\mathcal{Q}$ is a subset of the queries accepted by the external query interface. The precise definition of $\mathcal{Q}$ varies based on the characteristics and capabilities of the external query interfaces. A *querying policy* (*policy*) is a mapping $\pi : \mathcal{E} \rightarrow \mathcal{Q}$. To our knowledge, policies are traditionally written manually.

Example 2. Given e = *Zoloft*, the mediator must devise a keyword query to extract $X(e)$ = *Sertraline*. One policy is to use the content of the input entity (*Zoloft*) within the output mediator query. However, the content in `Brand` and `Approved Use` are likely unique to the local dataset. Given this observation, assume the mediator's policy ignores terms from `Brand` and `Approved Use` and prefers terms from `Drug Class` and `Description`. Thus, this policy maps e (*Zoloft*) to the keyword query *"serotonin depression panic"*.

Query Result. External query interfaces usually return results of query q as a list of entities inverse sorted based on the degree by which the query interface deems the entities relevant to q. More precisely, the *result* of query $q \in \mathcal{Q}$, $D[q]$, is a list of entities in D.

Query Effectiveness. Ideally, we would like the mediator query q submitted for local entity e to return the external entity relevant to e, i.e., $X(e)$ is placed in a relatively high position in $D[q]$. Given mediator query q for local entity e, we define the effectiveness of q over external dataset D as a real-valued function $f(X(e), D[q])$ whose range is in $[0, 1]$. The precise mechanics of f depends on the domain. For instance, there are standard metrics in information retrieval and data management to measure how effectively queries achieve this goal given their returned results [37]. For example, *precision@k* is the fraction of relevant answers in the top-k returned results. Another frequently used metric is *reciprocal rank* (RR) $\frac{1}{r}$ where r is the position of the first relevant answer. One metric may be more appropriate than another for a specific setting. For instance, reciprocal rank may be a better indication of effectiveness than precision@k if there are at most a couple relevant answers to the query. One can choose f based on the domain. In this paper, we use reciprocal rank.

Example 3. The mediator submits q = *"serotonin depression panic"* to the query interface over the external dataset in Figure 1, which returns the ranked results $D[q]$ = (*Paroxetine*, *Sertraline*). Since $X(e)$ = *Sertraline*, the reciprocal rank of these results would be $\frac{1}{2}$.

Effectiveness of Policy. A mediator's policy is evaluated based on the effectiveness of the queries it produces. More formally, the effectiveness of policy π for local dataset $\mathcal{E}$ and external dataset D is $F(\mathcal{E}, D, \pi) = \sum_{e \in \mathcal{E}} P(e)\ f(X(e), D[\pi(e)])$ where $P(.)$ is the prior probability of choosing local entities for augmentation by users. Unless otherwise noted, we assume that P(.) is uniform.

Optimal Policy. Optimal policy delivers the maximum effectiveness across the entire local dataset. More precisely, the optimal policy π^* for local dataset $\mathcal{E}$ and external dataset D is $\operatorname{argmax}_{\pi} F(\mathcal{E}, D, \pi)$.

Problem Statement. Given local dataset $\mathcal{E}$ and external dataset D, the mediator seeks to find the optimal policy π^* from the set of all possible policies. Finding the optimal policy requires a sufficient understanding of both the representation of entities in $\mathcal{E}$ and how query interface over D answers queries.

Merging Local and External Information. One might have to merge local data with its relevant external data by performing other steps of data integration, such as schema matching [14]. However, it takes more than one paper to investigate all steps of data integration. Thus, we assume that in these settings, users leverage existing data integration tools to create the final dataset and focus on the task of collecting information from external sources effectively.

Sudowoodo: Contrastive Self-supervised Learning for Multi-purpose Data Integration and Preparation

Runhui Wang[§]
Computer Science Department
Rutgers University
New Brunswick, United States
runhui.wang@rutgers.edu

Yuliang Li
Megagon Labs
Mountain View, United States
yuliang@megagon.ai

Jin Wang
Megagon Labs
Mountain View, United States
jin@megagon.ai

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00391

***Abstract*—Machine learning (ML) is playing an increasingly important role in data management tasks, particularly in Data Integration and Preparation (DI&P). The success of ML-based approaches, however, heavily relies on the availability of large-scale, high-quality labeled datasets for different tasks. Moreover, the wide variety of DI&P tasks and pipelines oftentimes requires customizing ML solutions at a significant cost for model engineering and experimentation. These factors inevitably hold back the adoption of ML-based approaches to new domains and tasks.**

In this paper, we propose Sudowoodo**, a multi-purpose DI&P framework based on contrastive representation learning.** Sudowoodo **features a unified, matching-based problem definition capturing a wide range of DI&P tasks including Entity Matching (EM) in data integration, error correction in data cleaning, semantic type detection in data discovery, and more. Contrastive learning enables** Sudowoodo **to learn similarity-aware data representations from a large corpus of data items (e.g., entity entries, table columns) without using any labels. The learned representations can later be either directly used or facilitate fine-tuning with only a few labels to support different DI&P tasks. Our experiment results show that** Sudowoodo **achieves multiple state-of-the-art results on different levels of supervision and outperforms previous best specialized blocking or matching solutions for EM.** Sudowoodo **also achieves promising results in data cleaning and column matching tasks showing its versatility in DI&P applications.**

I. Introduction

Machine learning, particularly deep learning (DL), is playing an increasingly important role in almost all fields in computer science including data management [1], [2], [3], [4]. More recently, this trend has extended to Data Integration and Preparation (DI&P) [5], [6], [7], [8] with learning-based solutions making promising progress and advancing the state-of-the-art in multiple tasks, including entity matching [9], [10], data cleaning [11], [12], [13], [14], and table annotation [15], [16], [17]. For example, Entity Matching (EM) solutions based on pre-trained language models such as BERT [18] achieve new state-of-the-art results across multiple matching tasks [19], [10]. While shown to be effective, as we describe next, existing learning-based solutions for DI&P suffer from two major challenges making them less attractive in practice.

Challenge 1: Label Requirement. The success of ML-based approaches comes at the cost of creating large-scale, high-quality annotated datasets. For example, in Entity Matching, learning-based methods typically require around 10k or more labeled entity pairs to achieve the best pairwise matching performance [9], [10]. Creating high-quality datasets of such cardinality can be quite expensive considering the required human efforts, which make ML-based solutions less accessible to new DI&P tasks and domains.

Challenge 2: Task Variety. There is a wide scope of DI&P tasks requiring ML models for different problem formulations. This means that oftentimes, practitioners have to build a specialized ML solution for each task, resulting in extra costs of model engineering. For example, Entity Matching typically consists of two tasks, *blocking* and *matching* [20], with the goal of (1) filtering out candidate matching pairs and (2) performing the pairwise comparison, respectively. The two tasks are closely related, but because of their different problem formulations, existing studies typically propose specialized solutions [21], [9], [19], [22], [10] that require separate modeling, data annotation, and experimentation efforts. for each task. This has also been the case for other DI&P tasks in data cleaning and data discovery [11], [12], [13], [14], [15], [16], [17].

To address these two challenges, we proposed Sudowoodo, a multi-purpose DI&P framework based on *contrastive representation learning* [23]. Sudowoodo features a generic DI&P problem definition of data matching: given a collection of data items (e.g., entity entries, table columns, cell values), identify all pairs of compatible items. This generic definition allows Sudowoodo to support a wide range of tasks in data integration, cleaning, and discovery. As Figure 1 illustrates, EM can be formulated as a task of matching entity entries; error correction in data cleaning [12], [13], [14] can be formulated as matching erroneous cell values with candidate corrections; and column matching, which is widely used in data discovery [24], [15], can be formulated as matching table columns that have the same semantic type. The matching criteria are also customizable w.r.t the specific application. In this work, we consider any criteria that can be cast into a general binary relation, such as whether two entity entries refer to the same real-world entity for EM.

Sudowoodo addresses the label requirement challenge by

[§]This work was done during the internship at Megagon Labs

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00391

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:56:35 UTC from IEEE Xplore. Restrictions apply.

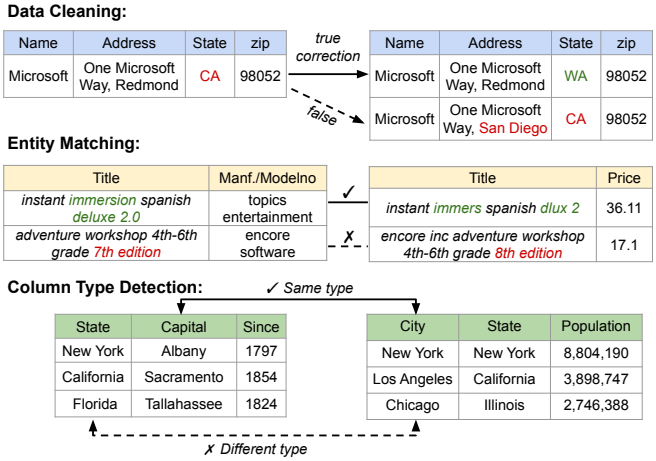


Fig. 1: Sudowoodo supports any DI&P tasks that can be formulated as a general form of matching task.

leveraging contrastive learning to learn a data representation model from a large collection of unlabeled data items, such as corpora of entity entries or table columns. Unlike that of language models such as BERT [18], the effectiveness of Sudowoodo's pre-training process relies on its contrastive learning objective instead of learning linguistic characteristics. The contrastive objective allows Sudowoodo to learn how to distinguish pairs of similar data items from dissimilar ones that are likely to be distinct. In this way, without using any labels, Sudowoodo learns meaningful representations where similar data items are close in the representation space while pairs of dissimilar data items are far apart.

After the pre-training process, the learned data representation model is accessible to downstream tasks either directly in an unsupervised manner or by *fine-tuning* using a few labels. Unlike that in language models, fine-tuning is not always necessary if the downstream task is similarity-based for which Sudowoodo already provides a reliable metrics space. When fine-tuning is needed, Sudowoodo provides an optional *pseudo labeling* step that extracts training signals with high confidence from the learned representations useful for further boosting the fine-tuning performance. As we will illustrate, pseudo labeling together with a customized fine-tuning architecture significantly improves Sudowoodo's performance on DI&P tasks in low-resourced settings.

Example 1: We illustrate the Sudowoodo pipeline in Figure 2 using Entity Matching (EM), a typical application of data integration. The upper part of the figure shows a standard EM pipeline consisting of the blocking and matching sub-tasks. Given two collections of entity entries (Tables A and B), the goal of blocking is to select a candidate set of matching pairs from the cross product to avoid the quadratic-size pairwise comparison. Next, it trains a matching model on a labeled dataset sampled from the candidate set, before applying the model to all the candidate pairs for the final matching results.

Sudowoodo starts with contrastive learning to learn the similarity-aware data representations of all entity entries from the two input tables. Because its model already captures entity similarity, we can solve the blocking task by applying

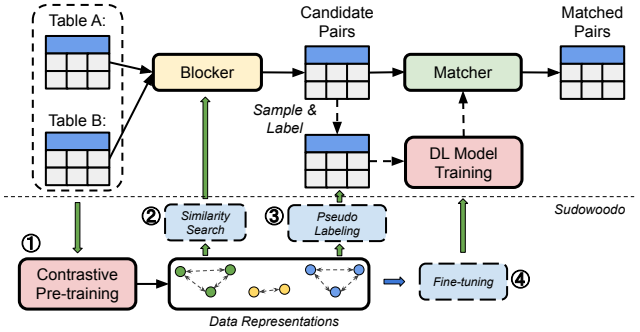


Fig. 2: The Sudowoodo pipeline for EM (blocking+matching).

nearest neighbor search to identify the top-k most similar entity entries for all input entries as the candidate set. For the matching task, practitioners can then fine-tune the pre-trained representation model on a labeled training set. For this step, Sudowoodo's pseudo labeling module allows automatic creation of additional high-quality labels by extracting match/unmatch pairs with high confidence from the learned representations. By doing so, Sudowoodo significantly reduces the label requirement for training the matching model.

Further optimizations. Based on the standard contrastive learning framework, Sudowoodo features several optimizations to further boost the pre-training effectiveness. First, the quality of learned representations heavily depends on the generated pairs of data items that are semantic-preserving. Sudowoodo obtain such pairs via data transformation such as synonym replacement or span deletion. To obtain more fine-grained transformations, Sudowoodo supports the *cutoff* data augmentation (DA) technique [25] that transforms input data items at the word-embedding level. Combining cutoff with a package of DI&P data augmentation operators, Sudowoodo obtains more diverse views for contrastive pre-training.

Another challenge for applying contrastive learning to DI&P tasks is how to obtain more effective negative examples. Negative examples allow the representation model to learn how to separate dissimilar data items. The standard contrastive learning approach uses uniformly sampled pairs as negative examples. This simple approach can be insufficient when random pairs fail to provide strong enough negative examples because they are easily separable. To address this issue, Sudowoodo splits the unlabeled data items into clusters and samples negative pairs within each cluster. By doing so, Sudowoodo allows the representation model to learn "harder" by distinguishing syntactically similar items.

In summary, this paper makes the following contributions:

- We propose a generic matching problem definition that captures a wide variety of DI&P tasks including (1) EM in data integration, (2) Error correction in data cleaning, (3) Semantic type detection in data discovery, and more.
- We propose Sudowoodo, a contrastive representation learning framework for the supported DI&P tasks. Sudowoodo pre-trains a representation model from a large collection of unlabeled data items. Practitioners can then apply the representations to downstream tasks either directly or by

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:56:35 UTC from IEEE Xplore. Restrictions apply.

fine-tuning with few labels.

- We introduce 3 optimizations to Sudowoodo. Specifically, we (1) augment the pre-training data using a novel cutoff operator, (2) introduce "harder" negative samples via clustering, and (3) combine the standard contrastive loss with a non-contrastive learning objective through redundancy regularization [26].
- We evaluate Sudowoodo on real-world datasets for EM, data cleaning, and semantic type detection. Sudowoodo achieves a series of new SOTA results under different label budgets, highlighting its capability of holistically providing solutions for multiple DI&P tasks.

The rest of the paper is organized as follows. Section II defines the problem and introduces the overall architecture. We introduce the contrastive learning algorithms in Section III and their optimizations in Section IV. Section V extends Sudowoodo to data cleaning and column matching. Section VI summarizes the experiment results. We discuss related work in Section VII and conclude in Section VIII.

II. Background and System Overview

In this section, we first define a general matching task for capturing a broad set of DI&P tasks including Entity Matching (EM), data cleaning, and column type detection. We then review the baseline method of pre-trained language models and fine-tuning for solving the task. We also provide an overview of Sudowoodo's architecture.

A. Task definition

In Sudowoodo, we consider DI&P tasks that can be formulated as a general problem of *matching related data items*. These data items can be of diverse formats across different DI&P tasks. For example, as Figure 1 illustrates, these data items can be entity entries, table columns, cell values, etc. The matching criteria can also be customizable binary relations w.r.t the specific DI&P applications, such as: whether two entity entries refer to the same real-world entity for EM; whether a row is a correct candidate correction to an erroneous row for data cleaning; or whether two table columns have the same semantic type for data discovery.

It is commonly seen that within a DI&P task, the same matching problem is formulated in different ways to address different application needs. For example, in an EM task, the blocking stage requires quickly getting a small candidate set that contains filtered matching pairs with high recall while the matching stage requires both high recall and precision. To facilitate their different needs, in Sudowoodo, we consider two flavors of the matching tasks: *embedding* and *pairwise matching*. The embedding task aims at generating a vector representation for each data item so that related data items are close in the vector space. The generated vectors can then be indexed and queried to support fast similarity search [27] or clustering [28]. The goal of the pairwise matching task is to perform binary classification for determining whether a pair of data items are related or not. We formally define these two tasks as follows:

Definition 1 (Embedding): Given a collection D of data items, a d-dimensional embedding model M_{emb} takes every data item $x \in D$ as input and outputs a real vector $M_{\mathsf{emb}}(x) \in \mathbb{R}^d$. Given a similarity function sim, e.g., cosine, for every pair of data items (x, x'), the value of $\mathsf{sim}(x, x')$ is large if and only if (x, x') matches.

For simplicity, we assume all output vectors are normalized, i.e. the L-2 norm $\|M_{\mathsf{emb}}(x)\|_2 = 1$ for all data item x.

Definition 2 (Pairwise Matching): Given a collection D of data items, a pairwise matching model M_{pm} takes a pair (x, y) of data items as input and outputs $M_{\mathsf{pm}}(x, y) \in \{0, 1\}$ where $M_{\mathsf{pm}}(x, y) = 1$ if (x, y) is a real match or 0 otherwise.

Note that we can also abuse the notation $M_{\mathsf{pm}}(x, y)$ as a 2-d real vector where each dimension indicates the predicted probability of non-match / match denoted as $M_{\mathsf{pm}}(x, y)_0$ and $M_{\mathsf{pm}}(x, y)_1$ respectively.

Entity Matching. With the help of Sudowoodo, it is rather easy to build an effective EM pipeline. The embedding model provides the similarity measure for the blocking stage of EM while the matching stage can directly use the pairwise matching model that Sudowoodo provides. For the rest of the paper, we will take EM as the main example when introducing the Sudowoodo framework for clarity. We will describe how to extend Sudowoodo to the data cleaning and column type detection tasks later in Section V.

B. Fine-tuning language models

The goal of Sudowoodo is to (1) train high-quality embedding models without labels and (2) train the pairwise matching models using no or just a few labels. Pre-trained language models (LMs), such as BERT, provide a good baseline solution which has recently achieved promising results for EM [10], [29], [22], [30]. These models typically consist of 12 or more Transformer layers [31] pre-trained on large unlabeled text corpora such as Wikipedia. In the process of pre-training, the LMs are trained in a self-supervised manner to perform auxiliary tasks such as missing token and next-sentence prediction. In this way, LMs effectively capture lexical or semantic meanings of the input text sequences. Formally, a LM M encodes an input token sequence x into a vector representation $M(x)$.

To apply pre-trained LMs in Sudowoodo, we first need to serialize the input data items into sequences of tokens. For EM, we follow the serialization method in Ditto [10] which converts an entity entry into the format:

[COL] Title [VAL] instant immers ... [COL] Price [VAL] 36.11

where [COL] and [VAL] are special tokens indicating the start of an attribute or value. We denote by $\mathsf{serialize}(x)$ the serialization of a data item x. We omit "serialize" when the context is clear. After that, we can obtain a baseline embedding model that first serializes an input data item and encodes it using the pre-trained LM.

Next, one can obtain a baseline pairwise matching model via *fine-tuning* the pre-trained model on a downstream task-specific training set. Pre-trained LMs like BERT support

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:56:35 UTC from IEEE Xplore. Restrictions apply.